Solar sec.

Project plan

PART 1 CONTEXT Does not count toward the 1,000-word limit

Intro / Why are we doing this

We started with the question that came up in the very first intensive class: how does the data about solar panel output actually arrive where it needs to be and how openly is this accessible?

To be more specific: solar systems all over Germany send their data back to the manufacturer of the inverter (often a company in China), so that the owner can watch the output of their system in an app. The data flow always goes from the panel to the provider and only then back to the user who actually owns the panels. So why can't we read that data locally, inside the house, and display it right there, instead of taking the detour through the manufacturer's cloud?

While this question came from pure curiosity (and, to be fair, having an actual microcontroller and Raspberry Pi to fiddle around with just sounds like a lot of fun), we actually think that there might be a market for this.

From interviewing people of the industry we know that homeowners and investors would like to have a display at home showing the data and are willing to pay a few 100 € for it. Sure, they can look at their phone, but renewable energy IS a lifestyle product for a lot of people! Having a trophy hanging on your wall that reminds you that you have PV on your roof is something people would want.

Functionality

The plan is to use the PV system that CODE already has (solar panels with a Hoymiles microinverter) to build a prototype for it. This lets us learn as much as possible and allows us to make as many mistakes as early as possible.

The idea is straightforward: A small screen connected to a tiny computer that hangs or sits somewhere at CODE and shows what the PV system on the roof is doing right now. It gets the data straight from the inverter over the home network, without the detour through the manufacturer's cloud. It only reads and stores data and has a few options of displaying them.

Everything involved in this process, besides the local network, belongs to us and there are many unknowns in the building process: The hardware had to be ordered so this part is set but we will need to figure things out as we go when it comes to the software: With 2 GB of RAM we need to figure out how much the operating system can be and how much functionality we will have. We will need to figure out which programming language we will use for our own software, and how to get OpenDTU onto the microcontroller (luckily, someone else already wrote that part).

PART 2 THE PLAN

1 System Overview

Figure 1 shows the whole setup at CODE. On top is the Hoymiles system, which was there before us. At the bottom is our system. In between sits the Wi-Fi router, and on the side balenaCloud, which only matters when there's an update.

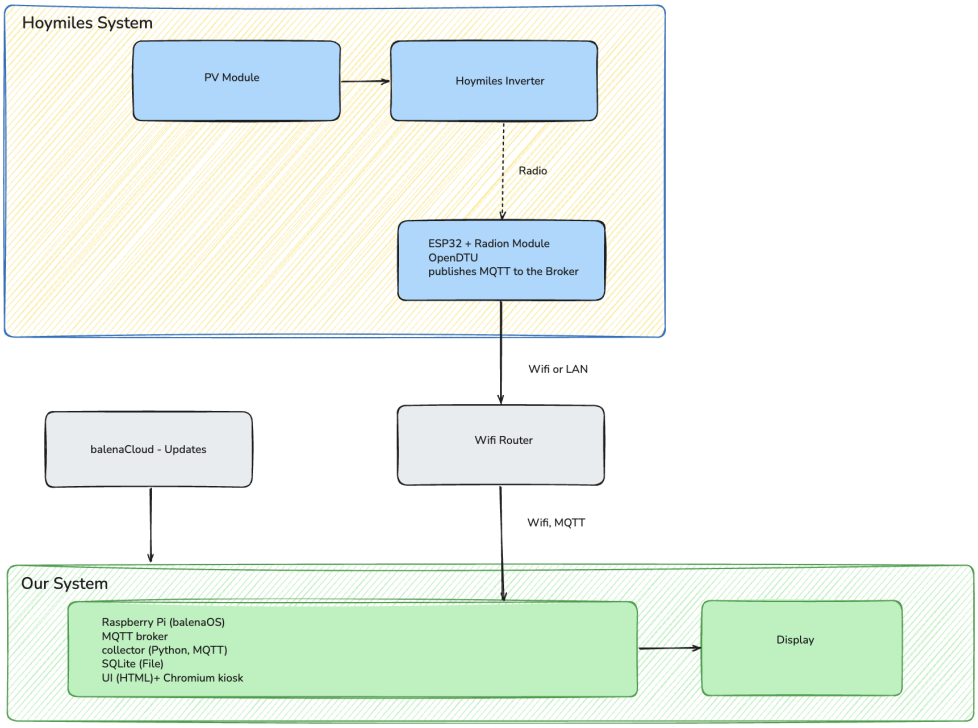


Figure 1. Network architecture of our project.

The Hoymiles side (producer of the solar panel)

The PV module sends its power to the Hoymiles inverter. The inverter measures everything, but it can only talk by radio. That's where our ESP32 microcontroller comes in. It's a special version of this controller that has an integrated radio module and runs OpenDTU, an open-source firmware that someone else already wrote (thankfully). OpenDTU talks to the inverter by radio and publishes the values as MQTT messages over the local network to the broker on our Raspberry Pi (All of this is part of the first intensive that we had with kit and if it seems complicated don't worry we really hope that we will be able to really understand it once we need to build it and write the code).

Our side

Our system runs on a Raspberry Pi 4 with 2 GB of RAM. Four things run on it:

- an MQTT broker that receives the messages
- a collector that picks them up and brings them into one format

- a small storage file
- a web page that runs in a full-screen browser and shows everything on the display

Updates come from balenaCloud, so we don't have to climb up to the device every time we change something. The Pi fetches them itself, which means no port has to be open to the internet. (This will be very important if we actually want to sell this. If we can't update it remotely we can't give it out to customers.)

Why is it split up like this?

Every arrow in the figure is a defined connection: radio, MQTT over Wi-Fi, a cable to the display. As long as we know what kind of connection it is, we can swap whatever sits behind it.

Sigenergy is a provider of actual PV modules for homes and is a good example. It doesn't need the ESP32 or the radio at all, because its Energy Controller is already on the network. We just ask it for values over Modbus TCP every few seconds. In our system, only the collector would change and the broker wouldn't be needed. Everything else stays the same.

Little fun fact which we learned about during the research: Modbus is an old, simple language from the world of industrial machines: the device keeps its values in numbered slots, and we basically just ask "what's in slot 30035?" and get a number back (in this case, how much power the panels are producing right now).

What we reuse and what we build

A lot of what we need, others needed before us: OpenDTU, the MQTT broker, balenaOS and the browser already exist. What we build ourselves is the collector and the display, because that's where our idea actually is: one display that works no matter which manufacturer is on the roof. Which language we write it in is still open. Python has the most ready-made pieces, but a compiled language like C would probably go easier on the 2 GB (remember the mind-blowing fact that Python is written in C, as Adam told us).

Constraints and what happens when things break

- **No internet (but Wi-Fi connection):** Everything keeps running, because the PV data never leaves the building. Only updates have to wait.
- **Pi is off:** OpenDTU has nobody to send its messages to, so there's a gap in the history.
- **Router is off:** OpenDTU and the Pi can no longer reach each other, so no new values arrive. The display keeps running and still shows the history, but it should say clearly that it has lost the connection, so nobody thinks the last value is still live. Once the router is back, everything should reconnect by itself. The daily yield is still right afterwards, because the inverter keeps counting it. Only the live curve has a gap.
- **Night:** The inverter goes to sleep. The display should say so instead of showing an error.
- **Who sees and decides what:** The owner sees the data, and whoever runs the updates controls the software. Nobody can change the PV system through our device, because it only reads.

2 Data Model

We are not only listing Hoymiles, the system we will use at CODE, but also trying to anticipate what other systems look like, since we want this product to theoretically work with all of them.

The Data Model needs to have a format so the incoming data needs to be transformed before it gets stored.

	Hoymiles	**Sigenergy (for comparison)**
Protocol	MQTT (OpenDTU)	Modbus TCP
Transfer	OpenDTU pushes to a broker and we listen. (Broker/publish-subscribe)	Data needs to be requested (polled)
Data flow	Hoymiles microinverters send data by radio. OpenDTU is open-source firmware for ESP32 microcontrollers. It receives the radio data and publishes it on the home network. The job of our system is to get the data from the home network, bring it into the right format and display it on the screen.	The Energy Controller speaks Modbus TCP itself. The collector connects to port 502, reads a set of registers at a fixed interval and converts the raw numbers.

If we want to set up the system for different systems, we always need to know what data arrives and how we have to convert it. The panel should only show the metrics where data is available.

3 Security

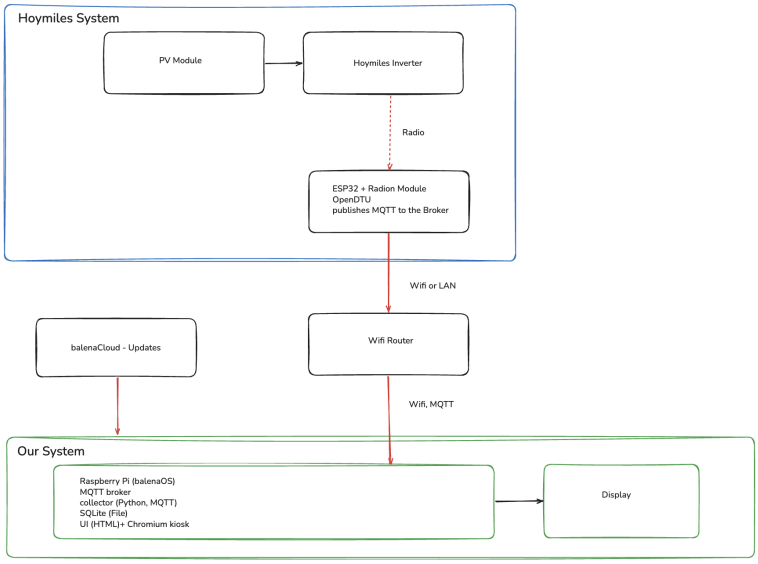


Figure 2. Interfaces in our system, marked in red.

Authentication and Authorization

Our system has no user accounts. The touchscreen has no login, because anyone at the screen already has physical access. The browser in kiosk mode has no browser controls.

Every interface is also a vulnerability surface. Interfaces are connecting the system objects. We use different interfaces in our system (red, Figure 2):

The radio connection is an interface which is out of our control. But it is only reachable in the range of the radio. The connection to balenaCloud is also an interface which is secured by device keys. Reachable through the home network is the MQTT broker on our device as well as the OpenDTU web interface. During the setup the web interface and broker can be secured by a login via username and password.

Anyone who can connect to the broker can read every message, but it is only reachable in our system. OpenDTU publishes to the broker on our device, and the collector subscribes to it. The broker has no cloud connection. Messages never leave the device. The broker listens only on the home network.

Reliability

When it comes to reliability we would add recoverability and independence to safety and enjoyability as metrics. The device should work after a power failure or Wi-Fi loss without someone visiting the site. The core functionality needs to be independent of a cloud service / internet connection.

Logging

What we would log

- **MQTT connection:** broker up or down, OpenDTU connected or lost
- **System state:** live, asleep (night) and offline
- **System health:** CPU temperature, free disk space, Wi-Fi signal
- **Storage:** write errors, database size, retention clean-up
- **Updates:** release changes and restarts
- **Security:** broker login failures, unexpected clients

What we do not log

- **Secrets:** MQTT password, balena keys, Wi-Fi password
- Customer names, addresses and Wi-Fi names
- Every raw message